Sonal saxena (1)

Artificial Intelligence (BCO019A) – V sem.

1950	1955	1955	1959	1961	1966	1974
AI incepted	Laws of Robotics	AI based S/w created	Self learning videogame	MIT founded AI lab	1st robot installed	1st chatbot Eliza

Unit I

Topic 1: What is Intelligence.

It is the ability to learn from experience, adapt to new situations, understand and handle abstract concepts, and use knowledge to manipulate one's environment. Intelligence, however is dependent on adaptation to the environment which involves :-

1. Perception
2. Learning
3. Memory
4. Reasoning
5. Problem Solving

Topic 2:- Foundations of AI

The foundation of AI includes various disciplines that contributed ideas, viewpoints and techniques to AI. Some of those disciplines are:-

1.) Philosophy 2.) Maths & Stats 3.) Economics 4.) Neuroscience
5.) Psychology 6.) CSE 7.) Control Theory & Cybernetics 8.) Linguisti-c

- **Philosophy** defines how can the formal rules be used to draw valid conclusions.
- **Maths & Stats** are important for writing algorithms, providing theorems, computations, decidability, learning from data, modeling uncertainity
- **Economics** deals with decision making in how to invest and where to invest
- **Neuroscience** deals with nervous system or rather how the brain process information.
- **Psychology** is the study of human vision, how you perceive something, how do people behave, how we think and act

- CSE is required for logics, algorithms, programming and system building. Sometimes it is hardware too. like GPU and TPU (Tensor Processing unit).
- Control theory helps the system to analyze, define, debug and fix errors by itself.
- Linguistics work as a speech techonology which enables a machine to understand the spoken language and translate into machine readable format. It is a way to talk to a computer. It shows how does a language relate to thought.

Topic 3 : Tasks of Artificial Intelligence.

The domain of AI is classified into different task categories such as:-

1.) Mundane / General Tasks :- The ones that we do on regular basis without any special training like common sense, reasoning, blaming etc.

2.) Formal Tasks :- Tasks where there is an application of formal logic or learning etc. like verification, theorem proving, games such as chess.

3.) Expert Tasks :- Tasks which require high analytical and thinking skills just like a professional. It could be scientific analysis, financial analysis, medical diagnosis.

Topic 4 :- Techniques of AI

The top 4 techniques of AI are :-

1) Machine learning :- making a machine learn from the environment.
2) Machine Vision :- collecting and analyzing the data which are utilized for processing and analyzing.
3) Natural Language Processing (NLP) :- Method of extracting information / meaning from human languages, comprehension

(4) Automation & Robotics :- Enabling machines to perform boring, repetitive jobs, increasing productivity and delivering more effective, efficient and affordable results.

Topic 5 :- Problem Solving in AI

The process of problem solving is frequently used to achieve objectives or resolve particular situations. It refers to A.I. methods which may include formulating proper use of algorithm and conducting root - cause analysis.

There are 3 types of problems in A.I.

1.) Ignorable :- in which solution steps can be ignored. Ex theorem proving.
2.) Recoverable :- in which solution steps can be undone. Ex. 8-Puzzle
3.) Irrecoverable :- Solution steps can't be Undo, Ex. Chess

Following are the steps to solve a problem:-

1.) Problem Definition :- detailed specification of inputs and acceptable system solution.
2.) Problem Analysis :- Analyse the problem thoroughly.
3.) Knowledge Representation :- collect detailed information about the problem and define all possible techniques.
4.) Problem - Solving :- Selection of best techniques.

Component to formulate problem :-

1.) Initial State :- from which the problem starts towords a specific goal.
2.) Action :- functions applied / description of possible actions available to the agent.
3.) Transition :- It describes what each action does.
4.) Goal Test :- determines if the given state is a goal state.

(5) Path cost :- it assigns a numeric cost to each path that follows the goal.

* Note :- Initial state + actions + transition = state space.
A state space ~~of~~ a problem is a set of all states which can be reached from the initial state followed by any sequence of actions.

The state space forms a directed map/graph where "nodes" are the states, "actions" are the links between the nodes and "path" is the sequence of states connected by sequence of actions.

Example Problems

Toy Problem
(concise and has exact description)
Ex:- Ludo,

Real World Problem
(Practical problems with no exact description)

Let us understand the above all concept with a quick example.

8 Puzzle / Sliding block Problem

Here, we have 3x3 matrix with movable tiles numbered from 1 to 8 with a blank space. The tile adjacent to blank space can slide into that space. Here, the objective is to reach a specified goal state.

Now, we will do problem formulation:-

i) **States** :- It describes location of each numbered tile and a blank tile.

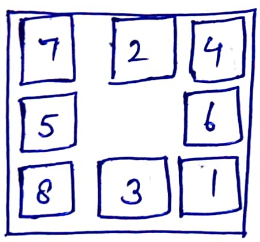
ii) **Initial State** :- We can start from any state as the initial state.

3.) **Actions :-** actions of the blank space is defined, either left, right, up and down.

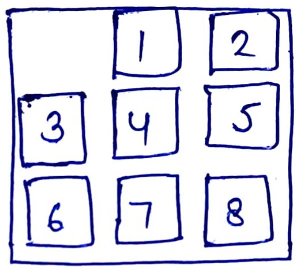
4.) **Transition model :-** it ~~result~~ returns the resulting states as per the given state and actions.

5.) **Goal Test :-** It identifies whether we have reached the correct goal state.

6.) **Path cost :-** no. of steps in the path where the cost of each step is 1.



Start state



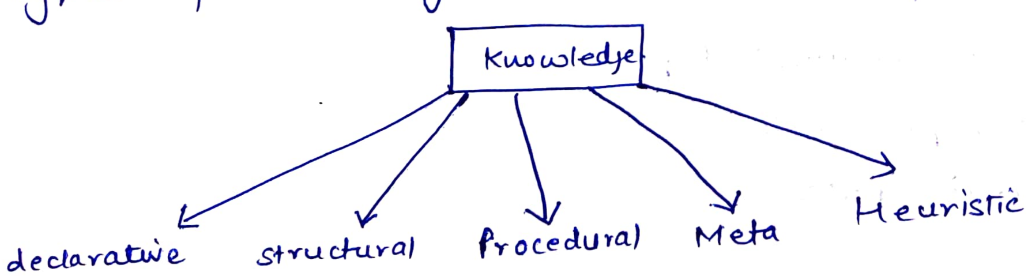
Goal state

Knowledge Representation

Topics

Knowledge Representation and Reasoning (KRR) is the part of AI for representing information so that a computer can understand and utilize this knowledge to solve the complex real world problems.

Types of knowledge :-



1.) **Declarative Knowledge :-**
- to know about something
- includes concepts, facts and objects
- simpler than procedural knowledge.

② **Procedural knowledge:-**

- also known as imperative knowledge.
- responsible for knowing how to do something.
- can be directly applied to task.
- includes rules, strategies, procedures & agendas
- depends on the task on which it can be applied.

③ **Meta knowledge:-**

- knowledge about other types of knowledge.

④ **Heuristic knowledge:-**

- representing knowledge of some experts in a file or subject.
- rule of thumb based on previous experiences, awareness which are good to work.

⑤ **Structural knowledge:-**

- basic knowledge to problem-solving.
- describes relationship b/w various concepts like, grouping of something.

Techniques for knowledge Representation

There are mainly four ways of knowledge representation which are as follows:-

i) Logical Representation
ii) Semantic Network Representation
iii) Frame Representation
iv) Production Rules.

:. Logical representation deals with propositional logic. It consists of syntax and semantics which supports sound inference. Each sentence can be translated into logics using syntax and semantics. It has 2 logics, propositional logic and predicate logic.

Semantic Network Representation is an alternative for predicate logic. In this presentation, we can represent our knowledge in the form of graphical networks. This network consists of nodes representing objects and arcs which describe the relationship between those objects. They are easy to understand and easily extensible. This representation consist of mainly 2 types of relations:-

i) IS-A-relation (Inheritance)

ii) Kind-of relation

Example:- following are some statements which we need to represent in the form of graphical n/w (nodes and arcs)

Statements:-

i) Jerry is a cat.

ii) Jerry is a mammal.

iii) Jerry is owned by Priya.

iv) Jerry is brown coloured

v) All mammals are animals.

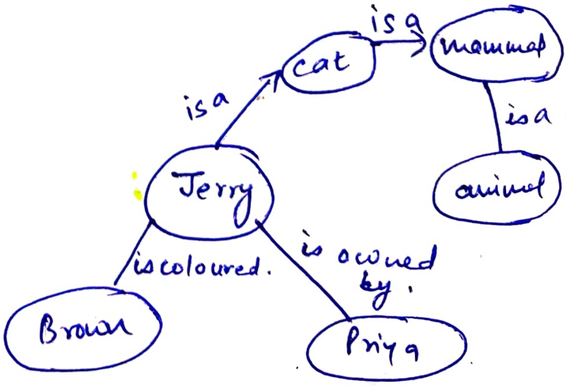


fig 1.

In fig 1 each object is connected with another object with a relation.

Frame Representation

A Frame is a record like structure which consists of a collection of attributes (properties) and its values to describe an entity in the world, Frames are collection of slot and slot values which may be of and type and size. Slots have names and values which are called facets. Facets are basically various aspects of a slot. They are features of frames which enable us to put constraints on the frames.

A frame may consist of any no. of slots, and a slot may include any no. of facets and facets may have any no. of values. It is widely used is NLP and machine vision. Example:-

Slots	Filter.
Name	Peter
Profession	Doctor
Age.	25
Marital Status	Single
weight-	78.

Production Rules.

Production rules system consist of (condition / action) pair which means "if condition then action". It has 3 main parts:-

i) set of production rules.
ii) working memory
iii) The recognize - act - cycle.

∴ The production rule checks for condition and if the condition exists; action is taken. The conditional part of the rule determines which rule may be applied to a problem.

The condition part of the rule determines which rule may be applied to the problem. And the action part carries out the associated problem-solved steps. This complete process is called recognize act cycle.

The working memory contains the description of the current state of problem-solving ~~rule~~ and rule can write knowledge to the working memory.

Example:-

1.) If (at bus stop AND bus arrives) THEN action (get into bus)

2.) If (on bus And unpaid) THEN action (pay charges)

Unit II - Uninformed & Informed Search Strategies.

Searching is the process in which a machine/AI finds the sequence of various steps required to solve the given problem. It is of two major types, Informed and Uninformed search in AI. Out of both, the informed search provides more efficiency and it costs less.

Informed Search (Heuristic)

The algorithms of informed search contain information regarding the goal state. It helps in more accurate and efficient searches. Example :- Graph Search & Greedy Search.

Uninformed Search (Blind)

The algorithms of these search, doesn't contain any additional data / info regarding the goal state. For example:- Breadth First Search, Depth First Search. It takes lot of time to generate solution of a problem.

Breadth First Search

It is a traversing/searching algorithm where you should start searching from a selected node (Source/Start) and traverse the graph layerwise/ levelwise; thus exploring the neighbouring nodes. It is a recursive algorithm. It uses "Queue" Data Structure to execute search Process.

Depth First Search

Again, It is also traversing/searching algorithms which is recursive in nature. It starts with initial node of graph and goes deeper until we find the goal node or the terminal node. "Stack" data structure is used to execute

Sonal Saxena

DFS, The implementation process is similar to BFS.

Depth Limited Search

It is uninfomed search process. It is similar to depth first search with a predetermined limit. It can solve the drawback of the infinite path in the Depth first Search. In this search, the node at the depth limit will treat as it has no successor nodes further.
It can be terminated with 2 conditions of failure :-

1) Standard failure value :- It indicates that problem doesn't have any solution.

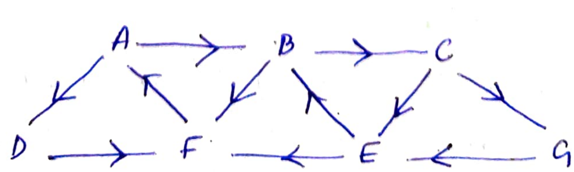
2) Cut-off failure value :- It defines no solution for the problem within a given depth limit.

Heuristic Functions in AI.

These functions are strategies or methods that guide the search processes in AI algorithms by providing estimates of the most promising path to a solution. They are often used in the scenarios where exact solution is computationally infeasible. It leads to a faster and more efficient approach of problem solving.
Some common Heuristic algorithms are :-

1) Hill Climbing Algorithm
2) Best First Search
3) A* algorithm
4) AO* algorithm

A : B D
B : C F
C : E G
D : F
E : B F
F : A
G : E



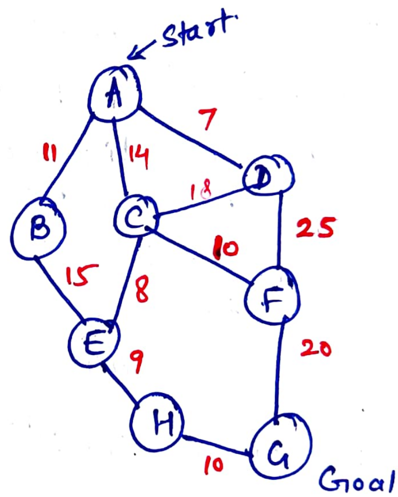
Best First Search (Informed Search Technique)

→ Combination of BFS and DFS
→ Has a greedy approach
→ uses an evaluation function to decide which node is most promising.
→ category of Heuristic Search.
→ Priority queue is used to store the cost of nodes.

Algorithm:-

A priority queue "PQ" containing initial states.

Loop
if PQ = Empty,
Return Fail
otherwise,
Node ← Remove First (PQ)
if Node = Goal
Return "Path from Initial to Node"
otherwise
Generate all successors of node & insert newly generated Node into PQ according to cost value.
End Loop.



"Here we are using "straight line Distance", which means directly measuring distance with a scale, from one node to another.

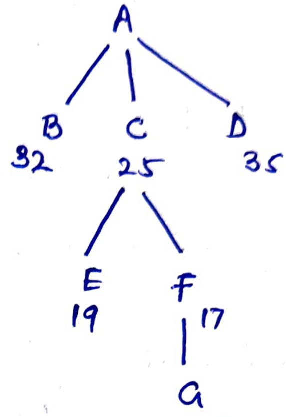
We will be taking 2 queues other than PQ, "OPEN" & "CLOSE". OPEN will hold the nodes to be processed and CLOSE will hold the already visited nodes.

A → G = 40
B → G = 32
C → G = 25
D → G = 35
E → G = 19
F → G = 17
G → G = 0
H → G = 10

Straight line Distance

So, as our initial node is A, we'll put it in OPEN list, and then remove it from OPEN to CLOSE as it is visited and we will insert its adjacent nodes in OPEN in "smallest cost first manner"

	open	close
i)	[A]	[empty]
ii)	[C B D]	[A]
iii)	[F E B D]	[A C]
iv)	[E B D]	[A C F]
v)	[G E B D]	[A C F G]



So, As we have reach goal state "G", we can trace the path, $A \rightarrow C \rightarrow F \rightarrow G$ and the cost would be summing their straight line distance.

$A \rightarrow C = 25$

$C \rightarrow F = 17$

$F \rightarrow G = 20$

$\Rightarrow 44$

Complexity :- $O(b^d)$, where b = branching factor and d = depth.

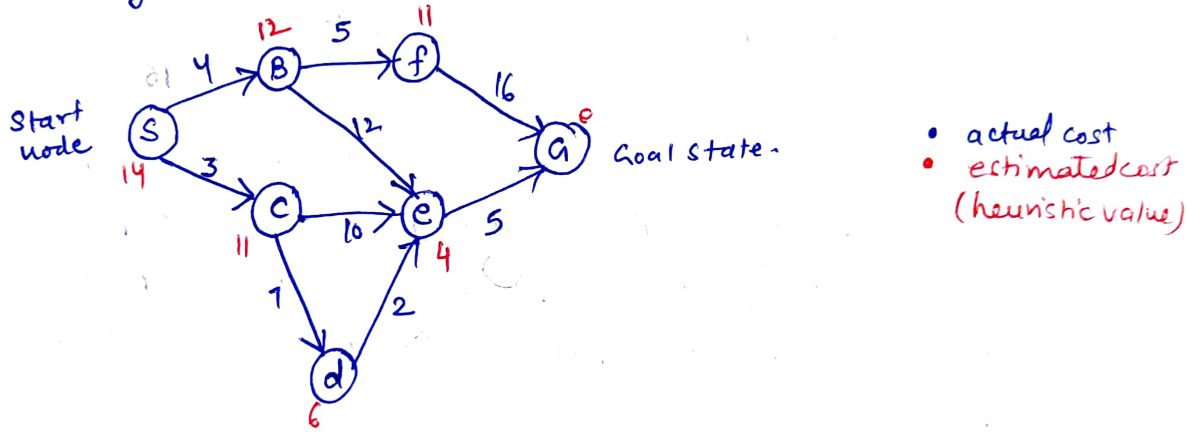
A^* Search Algorithm (Informed Search)

A* Search

- → works on Best First Search Algo.
- → Informed Searching Technique
- → Heuristic Search
- → uses $f(n) = g(n) + h(n)$ equation
- → "*" means admissible which gurantees optimal solution.

$$\underbrace{f(n)}_{\text{(Cost)}} = \underbrace{g(n)}_{\text{actual cost from start to goal}} + h(n) \text{ (estimated cost from start to goal node)}$$

- → Time Complexity - $O(V+E)$



Here, we need to start from node S and move to goal state with an objective of selecting the path which has the minimum cost. While transiting from start node to goal node, we need to use the "f(n)" equation. So let's first of all start from S → B and S → C

for S → B :-

actual cost $(g(n))$ $(SB) = 4$

estimated cost of $B(h(n)) = 12$

So, $g(n) + h(n)$

$= 4 + 12$

$= 16$

for S → C :-

actual cost$(g(n))$ $(SC) = 3$

estimated cost of $C(h(n)) = 11$

So, $g(n) + h(n)$

$= 3 + 11$

$= 14$

So, we can see S → C path is more promising as it has less cost in comparison to S → B cost. So, we proceed on S → C.

From SC, we can either go to e or d. Let's find out both the cost,

for SC → e :-

g(n) = g(n) [SC + CE]
= g(n) [SC] + g(n) [CE]
= 3 + 10
= 13

h(n) [e] = 4

f(n) = g(n) + h(n)
= 13 + 4
= 17

for SC → d :-

g(n) = g(n) [SC + CD]
= g(n) [SC] + g(n) [CD]
= 3 + 7
= 10

h(n) [d] = 6

f(n) = g(n) + h(n)
= 10 + 6
= 16

So, from above, we can see SC → d is more promising with less cost '16', but in the previous step we have got the same cost, S → B (16). So we can explore SB → F, SB → E or SCD → E. Let's calculate the cost of all there 3 routes :-

SB → F :-

⇒ g(n) + h(n)
⇒ (g(n) [SB] + g(n) [BF]) + h(n) [F]
= (4 + 5) + 11
= 9 + 11
= 20

SB → E

⇒ g(n) + h(n)
⇒ (g(n) [SB] + g(n) [BE]) + h(n) [E]
⇒ (4 + 12) + 4
⇒ 16 + 4
⇒ 20

SCD → E

⇒ g(n) + h(n)
= g(n)([SC]) + g(n)([CD]) + gn([DE]) + h(n) [E]
= (3 + 7 + 2) + 4
= 12 + 4 ⇒ 16

So; here comes the minimum cost, SCD→E = 16
we'll proceed towards goal from here. The next path would be SCDE → G.

$\Rightarrow$ $g(n) + h(n)$

$\Rightarrow$ $g(n)[SC] + g(n)[CD] + g(n)[DE] + g(n)[EG] + h(n)(G)$

$= 3 + 7 + 2 + 5 + 0$

$= 17$

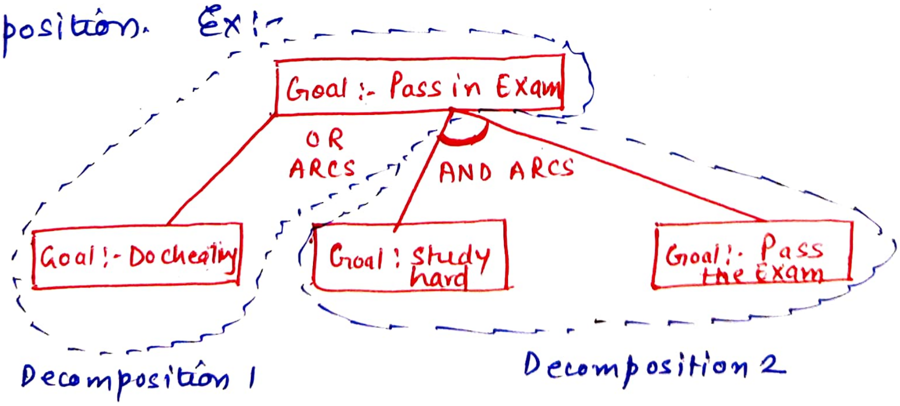
So, the minimum cost ~~from~~ of travelling from start node S to G is 17 with the path S → C → D → E → G.

AO* Algorithm for Searching.

→ Informed search
→ Based on Best First Search
→ uses AND-OR graphs
→ ~~~~ Doesn't always give optimal solution unlikely to A* algo.

About AND-OR graphs.

This graph is used for representing the solution of the problem that can be solved by decomposing them into a set of smaller sub problems, each of which must be solved. In this graph, OR node represents a choice between possible decompositions and AND node represents a given decomposition. Ex:-



Here, OR arcs gives a choice, that we can pass an exam either by cheating or by studying hard, but And arcs represents a single decomposition where in all the nodes needs to be processed that "study hard and pass the exam"

But again, here also the aim is to have minimum cost.

Let's now continue with AO* search Algorithm.

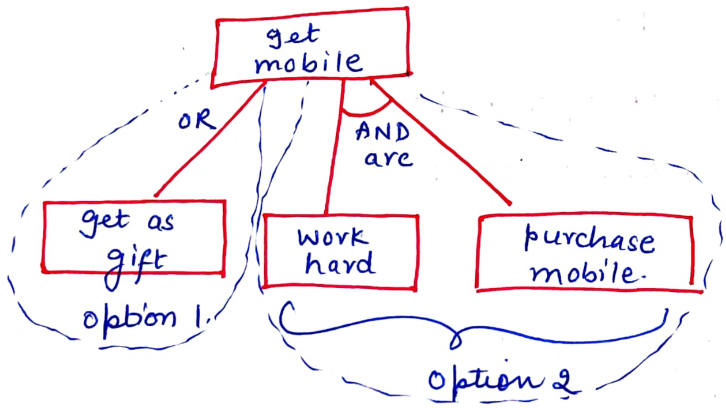
AO* Search Algorithm (Informed Search)

AO* Search

→ Stands for And-OR * / and-or graph.

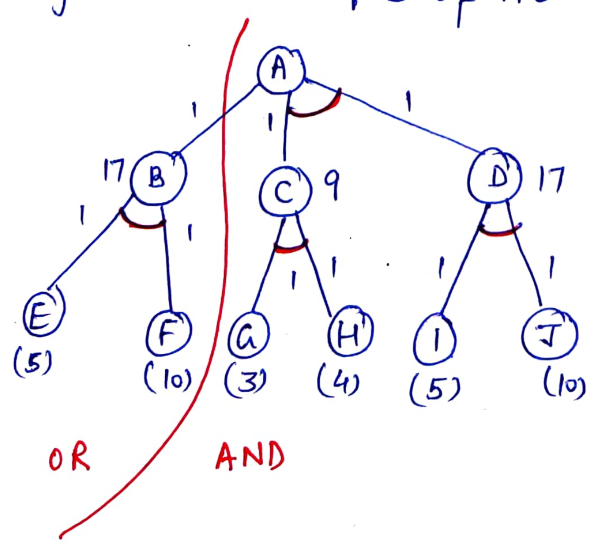
→ used for representing problems that can be solved by decomposing them into smaller problems and solving each of them.

Let us understand the problem first. Assume that 'X' wants to have a mobile. So there are 2 possibilities.



option 1 is that X can get mobile as a gift from some-one. But in option 2 he has to first work hard and then he can purchase the mobile. ~~Conc~~ Conclusively, we can say for option 2, both the conditions have to be fulfilled. So AND arcs have to be solved all the way, they are connected to each other. There could be 'n' successors in an AND arc.

Let's go through one example of AO* search algorithm



It's a bottom up approach, and we need to find out the least cost for the root node. Now starting from the last level / leaf node, we will calculate the cost of all the parent nodes of leaf node. Here, the heuristic cost of all leaf nodes is already given and we have assumed the cost of all links to be 1.

Cost of node B :- $h(E) + h(F) + 1 + 1 = 5 + 10 + 1 + 1 = 17$

Cost of node C :- $h(G) + h(H) + 1 + 1 = 3 + 4 + 1 + 1 = 9$

Cost of node D :- $h(I) + h(J) + 1 + 1 = 5 + 10 + 1 + 1 = 17$

Cost of A (with 'OR' option) :- $h(B) + 1 = 17 + 1 = 18$

Cost of A (with 'AND' option) :- $h(C) + h(D) + 1 + 1 = 9 + 17 + 2 = 28$

Now, as the cost 18 of 'OR' option is less than the 'AND' option, we will assign least value to node A. This is how AO* search algorithm works.

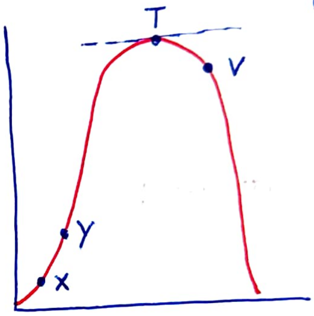
Hill Climbing search in A.I (Informed Search)

Hill Climbing search

- → It is a local search algo.
- → Uses greedy approach
- → No backtracking is done
- → variant of generate & test method
- → It always moves in a single direction.

Problem Statement :

Assume that there is a hill, and we want to climb it. Now, the current state for climbing the hill is 'X' right now. As we move upward to the next state 'Y', we need to check that whether 'Y' is a better state than 'X' or not. This comparison can be done by checking if the value of Y is greater than 'X' or not. If it is, then we will keep moving upward till we reach the top. But as soon as you reach on 'Top' (a stable position), the next state would be 'V', which will have value less than 'T'. In this case, if the next state value isn't greater than current value, the process will terminate.



This problem design can be applied to various problems. Eg 8 puzzle problem. Here also, the heuristic value has to be determined first. We can decide that ~~to~~ to proceed further, whether we want to have higher heuristic value or lower heuristic value. According we will compare and move forward.

Let's assume that we are on a specific state 'X' From here, I have 3 options viz: 4, 6, 3. Now, if I have decided that I will choose a heuristic value with a higher no. than I'll move forward with 6, and will discard 4 and 3. This means 4 and 3 will not be explored throughout the process at any cost ("No backtracking")

This also concludes that Hill Climbing is a DFS procedure.

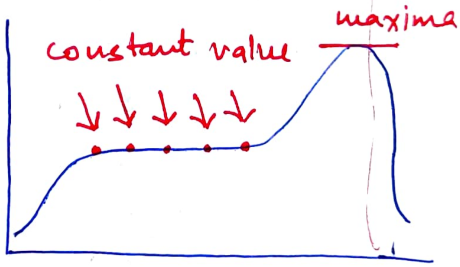
Sonal saxena Unit II (AI)

Limitations of Hill Climbing

1.) Local Maxima :-

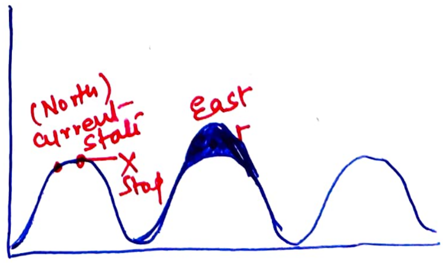
As we have already mentioned, that this algo uses greedy app-roach, so it only sees the local best option. It doesn't have information about the complete domain. So, once it reaches the top of the hill (local maxima) it will consider it as the best available solution without knowing that there is another hill (option) available which has far better solution than the current local maxima.

2.) Plateau :-



In this condition, the algo. get a same or a constant value for some time which gives an impression that there is no better value available, which is not true actually.

3.) Ridge :-



In this condition, the search process is moving forward towards a local maxima in a certain direction but somehow if we get to know that best result would be attained in some other direction or a slight deviation is required from the current direction, then again this process will terminate as it is not programmed that way.